

DOCUMENTAÇÃO TÉCNICA DO SISTEMA

Sistema de Barbearia

Item	Informação
Unidade Curricular	Programação Orientada a Objetos
Curso	Técnico em Informática
Estudante/Equipe	Arthur Pires, Murilo Furlan, João Vitor Lopes e Elison Gian
Turma	TI1-F
Docente	Lucas C. Machado
Versão do sistema	1.0
Data	30/09/2026

1. Identificação técnica do projeto

Item	Informação
Nome do sistema	Sistema de Barbearia
Versão	Versão inicial documentada
Linguagem	PHP 8.x
Ambiente de execução	PHP; página HTML5/CSS3 renderizada pelo navegador
Repositório GitHub	https://github.com/ArthurPires404/projetoBarbeariaGrupo-SEG.git
Branch da versão	main
Commit inicial	79d6e6f — Primeiro Commit do projeto
Data da versão documentada	30/09/2026

2. Visão técnica da solução

O Sistema de Barbearia foi desenvolvido em PHP com aplicação dos conceitos de Programação Orientada a Objetos. O domínio foi dividido em cinco classes principais: Pessoa, Cliente, Barbeiro, Servico e Agendamento. Pessoa concentra os dados comuns de uma pessoa e é utilizada como superclasse por Cliente e Barbeiro. Agendamento mantém referências tipadas a Cliente, Barbeiro e Servico para representar um atendimento completo.

O arquivo index.php funciona como ponto de entrada e demonstração do sistema. Nele são incluídos os arquivos das classes, criados objetos, exibidos seus dados, finalizado um atendimento e desativado um cliente. A apresentação visual é feita no próprio index.php com HTML e CSS.

3. Estrutura de diretórios e arquivos

Estrutura real da versão analisada:

Trabalho_Grupo_Barbearia/

- └─ Pessoa.php
- └─ Cliente.php
- └─ Barbeiro.php
- └─ Servico.php
- └─ Agendamento.php
- └─ index.php

Diretório/arquivo	Responsabilidade técnica
Pessoa.php	Define a classe base Pessoa, com nome, telefone, e-mail, construtor, getNome() e exibirDados().
Cliente.php	Define Cliente, que herda Pessoa e acrescenta preferência de corte e status ativo/inativo.
Barbeiro.php	Define Barbeiro, que herda Pessoa e acrescenta a especialidade do profissional.
Servico.php	Define Serviço, com nome, preço, duração e validação do preço.
Agendamento.php	Define Agendamento e associa Cliente, Barbeiro e Servico, além de controlar data/hora e conclusão.
index.php	Ponto de entrada: inclui as classes, instancia objetos e executa os fluxos demonstrados.

4. Diagrama de classes

Representação textual equivalente ao relacionamento implementado:

Pessoa

- └─ Cliente
- └─ Barbeiro

Agendamento — Cliente

Agendamento — Barbeiro

Agendamento — Servico

Pessoa é a superclasse de Cliente e Barbeiro. Agendamento possui atributos tipados como Cliente, Barbeiro e Servico, recebidos pelo construtor. Os atributos das classes analisadas são privados, e os métodos públicos controlam as operações disponibilizadas.

5. Dicionário de classes

5.1 Classe: Pessoa

Campo	Descrição
Arquivo	Pessoa.php
Responsabilidade	Representar os dados comuns de uma

	pessoa no sistema.
Superclasse	Não possui.
Classes relacionadas	Cliente e Barbeiro.
Observações	É a classe base utilizada por Cliente e Barbeiro.

Atributos

Atributo	Tipo	Visibilidade	Finalidade
\$nome	string	private	Armazena o nome.
\$telefone	string	private	Armazena o telefone.
\$email	string	private	Armazena o e-mail.

Métodos

Método	Parâmetros	Retorno	Finalidade
__construct()	string \$nome, string \$telefone, string \$email	void	Inicializa os dados da pessoa.
getNome()	nenhum	string	Retorna o nome da pessoa.
exibirDados()	nenhum	void	Exibe nome, telefone e e-mail.

5.2 Classe: Cliente

Campo	Descrição
Arquivo	Cliente.php
Responsabilidade	Representar o cliente e seu estado de conta.
Superclasse	Pessoa
Classes relacionadas	Pessoa e Agendamento.
Observações	Herda os dados e comportamentos de Pessoa.

Atributos

Atributo	Tipo	Visibilidade	Finalidade
\$preferenciaCorte	string	private	Armazena a preferência de corte.
\$ativo	bool	private	Indica se o cliente está ativo.

Métodos

Método	Parâmetros	Retorno	Finalidade
__construct()	string \$nome, string	void	Inicializa Pessoa, a

	\$telefone, string \$email, string \$preferenciaCorte		preferência e o status ativo.
desativar()	nenhum	void	Altera o status do cliente para inativo.
exibirDados()	nenhum	void	Exibe os dados herdados, preferência e status.

5.3 Classe: Barbeiro

Campo	Descrição
Arquivo	Barbeiro.php
Responsabilidade	Representar o profissional e sua especialidade.
Superclasse	Pessoa
Classes relacionadas	Pessoa e Agendamento.
Observações	Herda os dados e comportamentos de Pessoa.

Atributos

Atributo	Tipo	Visibilidade	Finalidade
\$especialidade	string	private	Armazena a especialidade do barbeiro.

Métodos

Método	Parâmetros	Retorno	Finalidade
__construct()	string \$nome, string \$telefone, string \$email, string \$especialidade	void	Inicializa Pessoa e a especialidade.
exibirDados()	nenhum	void	Exibe os dados herdados e a especialidade.

5.4 Classe: Servico

Campo	Descrição
Arquivo	Servico.php
Responsabilidade	Representar um serviço oferecido pela barbearia.
Superclasse	Não possui.
Classes relacionadas	Agendamento.
Observações	O preço é definido pelo método setPreco(), que trata valores negativos.

Atributos

Atributo	Tipo	Visibilidade	Finalidade
\$nome	string	private	Nome do serviço.
\$preco	float	private	Preço do serviço.
\$duracaoMinutos	int	private	Duração do serviço em minutos.

Métodos

Método	Parâmetros	Retorno	Finalidade
__construct()	string \$nome, float \$preco, int \$duracaoMinutos	void	Inicializa nome, preço e duração.
getNome()	nenhum	string	Retorna o nome do serviço.
getPreco()	nenhum	float	Retorna o preço.
setPreco()	float \$preco	void	Define o preço; se for negativo, armazena 0.0.
getDuracaoMinutos()	nenhum	int	Retorna a duração.
exibirDados()	nenhum	void	Exibe nome, preço e duração.

5.5 Classe: Agendamento

Campo	Descrição
Arquivo	Agendamento.php
Responsabilidade	Representar um atendimento associado a cliente, barbeiro e serviço.
Superclasse	Não possui.
Classes relacionadas	Cliente, Barbeiro e Servico.
Observações	O atributo concluido inicia como false e pode ser alterado por finalizarAtendimento().

Atributos

Atributo	Tipo	Visibilidade	Finalidade
\$cliente	Cliente	private	Armazena o cliente do agendamento.
\$barbeiro	Barbeiro	private	Armazena o barbeiro responsável.
\$servico	Servico	private	Armazena o serviço agendado.
\$dataHora	string	private	Armazena data e hora informadas.
\$concluido	bool	private	Indica se o atendimento foi

			concluído.
--	--	--	------------

Métodos

Método	Parâmetros	Retorno	Finalidade
__construct()	Cliente \$cliente, Barbeiro \$barbeiro, Servico \$servico, string \$dataHora	void	Inicializa as referências, data/hora e define concluido como false.
finalizarAtendimento()	nenhum	void	Altera concluido para true.
exibirDados()	nenhum	void	Exibe cliente, barbeiro, serviço, data/hora e situação.

6. Especificação dos métodos principais

6.1 Método: Servico::setPreco()

Item	Especificação
Assinatura	public function setPreco(float \$preco): void
Responsabilidade	Definir o preço do serviço.
Parâmetros/entrada	Um valor numérico do tipo float.
Retorno	void
Estado alterado	\$preco da instância.
Classes/objetos envolvidos	Objeto Servico.
Observações	Valores menores que zero fazem com que \$preco seja armazenado como 0.0.

6.2 Método: Agendamento::finalizarAtendimento()

Item	Especificação
Assinatura	public function finalizarAtendimento(): void
Responsabilidade	Marcar o atendimento como concluído.
Parâmetros/entrada	Nenhum.
Retorno	void
Estado alterado	\$concluido passa de false para true.
Classes/objetos envolvidos	Objeto Agendamento.
Observações	A situação exibida passa de Pendente para Concluído.

6.3 Método: Cliente::desativar()

Item	Especificação
Assinatura	public function desativar(): void
Responsabilidade	Desativar a conta do cliente.

Parâmetros/entrada	Nenhum.
Retorno	void
Estado alterado	\$ativo passa de true para false.
Classes/objetos envolvidos	Objeto Cliente.
Observações	A exibição posterior informa o status como Inativo.

7. Relacionamentos entre classes

Classe A	Classe B	Tipo de relação	Como aparece no código	Justificativa
Pessoa	Cliente	Herança	class Cliente extends Pessoa	Cliente é uma especialização de Pessoa.
Pessoa	Barbeiro	Herança	class Barbeiro extends Pessoa	Barbeiro é uma especialização de Pessoa.
Agendamento	Cliente	Associação por referência	private Cliente \$cliente; e parâmetro Cliente no construtor	O agendamento precisa de um cliente.
Agendamento	Barbeiro	Associação por referência	private Barbeiro \$barbeiro; e parâmetro Barbeiro no construtor	O agendamento precisa de um barbeiro.
Agendamento	Servico	Associação por referência	private Servico \$servico; e parâmetro Servico no construtor	O agendamento precisa de um serviço.

8. Herança, generalização e especialização

A herança está implementada nas classes Cliente e Barbeiro por meio de extends Pessoa. Ambas chamam parent::__construct() para inicializar os atributos comuns nome, telefone e e-mail. Depois, cada subclasse inicializa seu próprio atributo específico: preferência de corte em Cliente e especialidade em Barbeiro.

9. Sobrescrita e polimorfismo

Elemento	Classe(s) envolvida(s)	Método/comportamento	Explicação técnica
Sobrescrita	Pessoa → Cliente	exibirDados()	Cliente redefine exibirDados() e chama parent::exibirDados()

			antes de apresentar preferência e status.
Sobrescrita	Pessoa → Barbeiro	exibirDados()	Barbeiro redefine exibirDados() e chama parent::exibirDados() antes de apresentar a especialidade.

O código demonstra claramente sobrescrita de método. Embora isso faça parte do polimorfismo em POO, o index.php chama diretamente os objetos concretos (\$cliente1 e \$barbeiro1), não havendo no fluxo analisado uma variável declarada como Pessoa que receba objetos de subclasses e seja usada para demonstrar uma chamada polimórfica explícita.

10. Fluxos técnicos do sistema

10.1 Fluxo: criação e exibição dos objetos

index.php

↓

new Cliente(...), new Barbeiro(...), new Servico(...)

↓

new Agendamento(\$cliente1, \$barbeiro1, \$servico1, \$dataHora)

↓

exibirDados() dos objetos

↓

Dados apresentados na página

No index.php são criados os objetos com dados de exemplo e, em seguida, seus métodos exibirDados() são chamados para apresentar as informações.

10.2 Fluxo: finalização do atendimento

\$agendamento1->finalizarAtendimento()

↓

\$concluido = true

↓

\$agendamento1->exibirDados()

↓

Situação: Concluído

10.3 Fluxo: desativação do cliente

\$cliente1->desativar()

↓

\$ativo = false

↓

\$cliente1->exibirDados()

↓
Status: Inativo

11. Regras e validações implementadas

ID	Regra/validação	Classe responsável	Método	Tratamento implementado
R01	Preço não pode permanecer negativo.	Servico	setPreco()	Se \$preco < 0, \$this->preco recebe 0.0.
R02	Novo cliente inicia ativo.	Cliente	__construct()	\$this->ativo recebe true na construção.
R03	Cliente pode ser desativado.	Cliente	desativar()	\$this->ativo recebe false.
R04	Novo agendamento inicia pendente.	Agendamento	__construct()	\$this->concluido recebe false.
R05	Atendimento pode ser finalizado.	Agendamento	finalizarAtendimento()	\$this->concluido recebe true.

12. Dependências entre arquivos/classes

Origem	Dependência	Motivo
Cliente.php	Pessoa.php	require_once e herança de Pessoa.
Barbeiro.php	Pessoa.php	require_once e herança de Pessoa.
Agendamento.php	Cliente.php	require_once e tipagem do atributo/parâmetro Cliente.
Agendamento.php	Barbeiro.php	require_once e tipagem do atributo/parâmetro Barbeiro.
Agendamento.php	Servico.php	require_once e tipagem do atributo/parâmetro Servico.
index.php	Pessoa.php, Cliente.php, Barbeiro.php, Servico.php, Agendamento.php	require_once das classes usadas na execução.

13. Casos de teste

Os testes abaixo correspondem aos fluxos efetivamente executados pelo index.php.

ID	Cenário/entrada	Operação	Resultado esperado	Resultado obtido	Situação
T01	Cliente João Silva	Criar Cliente e exibir dados	Dados do cliente apresentados	Dados apresentados ; status inicial	APROVADO

			e status Ativo.	Ativo.	
T0 2	Barbeiro Marcos	Criar Barbeiro e exibir dados	Dados e especialidade apresentados .	Dados e especialidade apresentados .	APROVADO
T0 3	Serviço Corte Degradê + Barba, R\$ 50,00, 45 min	Criar Serviço e exibir dados	Nome, preço e duração apresentados .	Nome, preço e duração apresentados .	APROVADO
T0 4	Agendamento em 30/09/2026 às 14:00	Criar Agendamento e exibir dados	Cliente, barbeiro, serviço, data/hora e situação Pendente apresentados .	Informações apresentadas ; situação Pendente.	APROVADO
T0 5	Agendamento criado	Executar finalizarAtendimento()	Situação alterada para Concluído.	Situação exibida como Concluído.	APROVADO
T0 6	Cliente João Silva	Executar desativar()	Status alterado para Inativo.	Status exibido como Inativo.	APROVADO
T0 7	Preço negativo	Executar setPreco() com valor < 0	Preço armazenado como 0,00.	Regra implementada no método; teste específico não é executado pelo index.php.	A validar

14. Histórico técnico de versões

Versão	Data	Conteúdos incorporados	Principais alterações
1.0 / Inicial	30/09/2026	Pessoa, Cliente, Barbeiro, Serviço, Agendamento e index.php	Primeiro commit do projeto, com organização das classes, herança, associações e demonstração dos fluxos no index.php.

15. Limitações técnicas conhecidas

- O sistema analisado é uma demonstração em PHP e não apresenta persistência em banco de dados.
- Os dados utilizados no index.php são definidos diretamente no código como exemplos.
- Não há mecanismo de autenticação ou autorização implementado nos arquivos analisados.
- A data e hora do agendamento são armazenadas como string, sem validação específica de formato ou disponibilidade.
- A interface visual está concentrada no index.php, com HTML e CSS incorporados ao próprio arquivo.

16. Possíveis evoluções técnicas

- Adicionar banco de dados para persistência de clientes, barbeiros, serviços e agendamentos.
- Separar a camada visual da lógica PHP em arquivos independentes.
- Implementar validação de data, horário e disponibilidade do barbeiro.
- Adicionar autenticação e níveis de acesso.
- Criar operações de cadastro, alteração, consulta e exclusão com persistência.
- Adicionar testes específicos para regras como preço negativo.

17. Evidências da versão final

Evidência	Informação
URL do repositório	A preencher com a URL do GitHub.
Branch/versão final	main
Commit final conhecido	79d6e6f — Primeiro Commit do projeto.
Data da validação	30/09/2026 — validação baseada nos arquivos enviados.

18. Declaração de revisão da documentação

- ☒ O diagrama corresponde às classes implementadas.
- ☒ Os atributos e métodos documentados foram conferidos nos arquivos PHP enviados.
- ☒ Os relacionamentos descritos podem ser identificados na implementação.
- ☒ Os conceitos de herança, encapsulamento e sobrescrita citados foram identificados no código.
- ☒ Os fluxos principais registrados foram conferidos no index.php.
- ☒ Os testes específicos que não são executados pelo index.php devem ser executados antes da entrega.

☒ O README.md e esta documentação devem apontar para a mesma URL/versão final do repositório.

☒ Conteúdos produzidos com apoio de IA devem ser revisados pela equipe antes da entrega.